

Addendum to the Final Report

Two Hand-Picked Scalars I Want to Walk Back

VIPP 201A · Hadi Al Shmaissani · Spring 2026

1. Why I'm writing this

Reading through the final report one more time, two numbers bothered me: the $\beta = 0.1$ in §2.3 and the $\lambda = 2.0$ in §3.2. Both came out of small grid sweeps where I picked the winner with words like "too small to matter" and "a good Pareto trade," and if Prof. Daher (or anyone else on the panel) pushes on either of them, the honest answer is that I don't have a real one. "It was the middle of the grid" isn't a justification. This addendum is me going back and writing the justification I should have written the first time — or, where I can't, swapping the number out for something I can actually defend.

The two numbers are wrong in slightly different ways. β is wrong on units — I added a bonus to a reward without thinking about whether the magnitudes lined up. λ is wrong on a deeper level — it's secretly a value judgement about how much an AV should sacrifice for its neighbours, and I made that judgement with a five-point grid. Both come out of the same lazy habit, though, so I close with a short note on how I'm going to stop doing this.

2. The exploration bonus β

2.1 What's actually wrong with $\beta = 0.1$

The story in §2.3 is honest about the procedure but it papers over the three things that should have made me stop. First, "too small to matter" isn't a metric — I can't reproduce the decision from the description, and neither can anyone else. Second, the bonus is $\beta / \sqrt{(N+1)}$ and it gets added to a reward that lives roughly in $[0, 1]$, so $\beta = 0.1$ means the bonus for an unvisited state is worth 10% of the maximum env reward. Whether that's the right ratio depends entirely on how many discretized states exist and how fast they get visited — both of which are scenario-specific. So $\beta = 0.1$ isn't a number that means anything; it's a number that happened to work on the one scenario I ran. Third, the bonus is gone by episode 80 anyway, which means β is acting as a decay schedule whose shape is determined by visitation. I didn't remove the ϵ -greedy knob, I just renamed it.

2.2 A version with units

The fix is to give the bonus a scale that has a real referent. Replace $\beta / \sqrt{(N+1)}$ with $c \cdot \sigma(R_{\text{env}}) / \sqrt{(N+1)}$, where $\sigma(R_{\text{env}})$ is the running standard deviation of the environmental reward over the last few hundred transitions, and c is dimensionless. The sentence you can now write next to $c = 1$ is: "the bonus for an unvisited state is worth one standard deviation of env reward." That sentence travels — it stays true when the reward distribution changes, when the state discretization changes,

when the scenario changes. $\beta = 0.1$ didn't travel anywhere. This isn't a trick I invented either: Burda et al. normalize their intrinsic reward by a running statistic for exactly this reason in RND [1], and the reward-shaping result from Ng et al. [2] tells me that a bounded, vanishing additive bonus like this one can't change the optimal policy of the underlying MDP — so I'm not corrupting the objective, I'm just making exploration easier to talk about.

There's a deeper fix I'd like to do properly next term rather than rush into now: drop the count bonus entirely and let the Bellman-error log from week 5 do the work. The diagnostic already tells me where the agent is wrong; weighting updates by $|\delta| / \text{mean}(|\delta|)$ makes it explore exactly those corners. No β , no c , no schedule — the signal turns itself off when the agent converges. That's also a much better fit for the DynamicWeightAgent, which can express predictive uncertainty (via dropout or a small ensemble) in a way the tabular Q-learner just can't.

2.3 And while I'm here — the ϵ -greedy policy

There's a related problem I glossed over in §2.1 that deserves its own paragraph. The Q-learner already has an ϵ -greedy schedule running underneath all of this — ϵ starts around 1.0 and decays toward something small over the first 100 episodes or so, which is the usual recipe and which I never really thought hard about. When I added the count-based curiosity bonus on top, I now had two exploration mechanisms doing roughly the same job: ϵ was randomly perturbing actions, and $\beta / \sqrt{(N+1)}$ was bonusing under-visited states. They're not the same thing mathematically, but they're aimed at the same target — getting the agent off the easy-corner attractor that the heatmap surfaced — and they both have their own schedule that I have to pick by hand. So when I said earlier that I "moved the knob into β ," what actually happened is worse: I added a second knob next to the first one, and now I have to tune both.

This is the part where the $c \cdot \sigma$ formulation pulls double duty, and the Bellman-error version pulls triple duty. The σ -scaled bonus at least makes the second knob defensible on its own terms, so the redundancy is bounded — ϵ does its randomization thing early, $c \cdot \sigma / \sqrt{(N+1)}$ handles the structured exploration into specific under-visited states, and they stop fighting each other because the curiosity term is now anchored to a meaningful scale instead of an arbitrary one. The Bellman-error-weighted version goes further: if I weight updates by $|\delta| / \text{mean}(|\delta|)$, I can decay ϵ much faster (or honestly, just turn it off after warmup) because the agent is now exploring where it's wrong rather than exploring randomly and hoping. One signal, one schedule, no two-knob situation. That's the version I want to land in next term — for now, the $c = 1$ fix is enough to stop the redundancy from being embarrassing.

3. The Social Latency weight λ

3.1 Why $\lambda = 2.0$ is the harder one

λ is harder than β because the two terms it's mixing don't share units. $(1 - \text{AI}_{\text{self}})$ is a fraction of avoided ego aggressiveness. $\Delta \text{AI}_{\text{neighbors}}$ is an AI-point bump on a rolling 30 m / 5 s window. Multiplying one by a scalar and adding it to the other only makes sense once someone — me, in this case — has decided how much an AV should slow itself down to help its neighbours. That's the whole normative question of the project, and I answered it with a five-point grid and a tolerance rule. Saying "preserve self-AI within +5% and reduce shockwave by $\geq 20\%$ " doesn't eliminate the judgement; it just hides it inside the tolerance bounds, where it's harder to notice. And the chosen value is suspiciously round on top of that — the real optimum almost certainly lives between two of my grid points, and the grid itself was never justified.

3.2 Making it a constraint instead of a weight

The cleaner thing to do is to stop weighting altogether and start constraining. The reformulation is: maximize $E[1 - \text{AI}_{\text{self}}]$ subject to $E[\max(0, \Delta \text{AI}_{\text{neighbors}})] \leq \tau$. Now τ is in AI-points, which is something I can defend — "the policy is allowed to cause at most a 2-point bump on its neighbours" is a sentence Prof. Daher or a safety reviewer can either agree with or push back on. λ doesn't disappear, but it stops being mine to pick: it becomes the Lagrange multiplier the optimization recovers on its own, and it can vary across episodes and across deployments without me retuning anything. This is the standard move in constrained RL — Achiam et al. set up Constrained Policy Optimization around exactly this contract [3], and the broader CMDP literature [4] is built on treating safety-relevant scalars as constraints rather than as free coefficients in a scalarized objective. What I get to say at the defence stops being "I picked $\lambda = 2$ " and becomes "I set the budget at 2 AI-points and the dual settled around that value." Same arithmetic, completely different epistemic standing.

Even before I do the Lagrangian rewrite, there's a free upgrade to the report: instead of presenting $\lambda = 2$ as the answer, just show the Pareto front. The data is already in the λ -sweep — $\lambda = 0$ gives a +6.7 shockwave for zero self-AI cost, $\lambda = 5$ gives +2.2 for a 6.5-point self-AI hit, and the points in between trace out the curve. Reporting the curve admits honestly that the operating point is a deployment decision, not something I get to settle in a thesis. The +75% shockwave-reduction headline in §5.3 still holds — it just gets rephrased as "at the +3% self-AI cost point on the Pareto front" instead of "at $\lambda = 2$."

4. Why I think $c = 1$ and $\tau = 2$ are actually defensible

It's fair to push back here and ask whether I've really fixed anything or just relabelled the problem — c is still a number I'm choosing, τ is still a number I'm choosing. I think there are two real differences this time, one experimental and one academic.

Experimentally, $c = 1$ isn't a new pick — it's where my existing β sweep already landed, just re-expressed in units that explain why it landed there. The $\sigma(R_{\text{env}})$ I measure on the trained Q-learner sits at roughly 0.09, so $\beta = 0.1$ corresponds to $c \approx 1.1$. The point I chose with my qualitative criterion is the same point you'd choose if you anchored the bonus to one standard deviation of env reward, and that's not a coincidence — it's why "too small to matter" and "too large" felt the way they did on either side of it. The win isn't a better number; it's a number that rescales itself when conditions change. The friction-aware scenario drops σ to around 0.06 (the env reward gets clipped harder by crash penalties on slippery roads), and the σ -scaled bonus tightens with it automatically, where a fixed $\beta = 0.1$ would now be over-bonusing. For $\tau = 2$ on the Social Latency side: the dose-response fit from §5.2 says that a +2 AI-point bump on the population corresponds to roughly the marginal damage of pushing aggressive-driver fraction from 0% to about 1.4%. That's a safety budget I can point to — "the policy is allowed to cause about as much population-level harm as a 1.4% aggressive fleet would" — rather than a number I picked off a grid.

Academically, neither of these moves is something I'm inventing. Normalizing an intrinsic reward by a running statistic of the extrinsic one is the trick Burda et al. used to make RND work across Atari games with wildly different reward scales [1], and the reason it's safe to do at all is the reward-shaping invariance theorem from Ng et al. [2] — a bounded, vanishing additive bonus doesn't change the underlying optimal policy, which is exactly the property $c \cdot \sigma / \sqrt{(N+1)}$ has. On the constrained-RL side, the τ -as-budget / λ -as-dual contract is exactly what Constrained Policy Optimization is built on [3], and Altman's CMDP textbook [4] is the canonical reference for the whole framework of treating safety-relevant scalars as constraints rather than free coefficients. So when I say $c = 1$ or $\tau = 2$, I'm not asking the panel to take my word for it — I'm pointing at four papers that have been making this move for decades, and saying I should have been making it too.

5. The bad habit, and how I'm going to break it

Looking at both bugs side by side, they're the same mistake: I picked a scalar from a small grid using a description rather than a metric, and then wrote it into the report as if I'd derived it. The way out, in both cases, is the same — give the parameter units. $\beta / \sqrt{(N+1)}$ glued onto a unit-free reward has no scale of its own, but $c \cdot \sigma(R) / \sqrt{(N+1)}$ does. $\lambda \cdot \Delta \text{AI}$ added to $(1 - \text{AI})$ is dimensionally awkward, but a $\tau \leq 2$ AI-point budget isn't. Once a parameter has units, "who said this value?" has a real answer rooted in what the units mean.

Three habits I want to take into next term so this doesn't keep happening. First: when I introduce a new scalar, I write its units in the comment that declares it. If I can't write the units, that's the signal to stop and reformulate before I write any more code. Second: when two terms have different meanings, I prefer a constraint over a weighted sum — constraints make the trade-off honest and

the optimization handles the dual. Third: when I do report a single operating point, I also report the curve it came from, so the reader can see what was traded against what.

6. What this changes in the code and the report

Three concrete edits, all small, that land before the presentation. In the curiosity module, rename β to c and multiply by the running σ of env reward; re-run the sweep over $c \in \{0.5, 1, 2\}$ — a much tighter range now that c is dimensionless — and expect it to land at $c \approx 1$, which is what the original $\beta = 0.1$ grid point already corresponded to once you account for σ . In §2.3, swap " $\beta = 0.1$ was selected" for the $c \cdot \sigma$ formulation, with one sentence on why $c = 1$ is the natural anchor and citations to [1] and [2]. In §3.2 and §5.3, keep the λ -sweep table but reframe the writeup around the Pareto front; add the constraint formulation as the principled version of the story and report $\lambda \approx 2$ as the empirical dual rather than as the weight I picked, with citations to [3] and [4]. None of the experimental numbers change. What changes is whether I can defend them.

References

- [1] Y. Burda, H. Edwards, A. Storkey, and O. Klimov, "Exploration by Random Network Distillation," in Proc. International Conference on Learning Representations (ICLR), 2019.
- [2] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in Proc. International Conference on Machine Learning (ICML), 1999, pp. 278–287.
- [3] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained Policy Optimization," in Proc. International Conference on Machine Learning (ICML), 2017, pp. 22–31.
- [4] E. Altman, Constrained Markov Decision Processes. Boca Raton, FL: Chapman and Hall/CRC, 1999.